

# **PREDICTING DEFAULT RISK OF LENDING CLUB LOANS**

By Ali Ahmad, Atharv  
Kothekar, Uzair Shahed Islam



# PROBLEM STATEMENT

- Lending decisions carry **significant financial risk**. To avoid costly defaults, institutions must accurately predict whether a borrower **will repay or fail** to meet obligations.
- This binary classification task uses **borrower, loan, and credit attributes** to determine the likely loan outcome.
- Improved prediction directly leads to **smarter lending decisions** and **fewer high-risk approvals**.

# DATASET

- The project uses the **Kaggle Lending Club Loan Data** covering loans from 2007–2018.
- The dataset contains **2,260,701 accepted loans** with **151 borrower, loan, and credit features**.
- The target variable is derived from loan\_status:
  - **1 — Non-default loans** (e.g., Fully Paid, Current)
  - **0 — Default-risk loans** (e.g., Charged Off, Late)

# PREPROCESSING - 1

- **Label construction:** Binary target from loan\_status: 1 for non-default (Fully Paid, Current), 0 for default-risk (Charged Off, Late, etc.)
- **Leakage removal:** Dropped post-origination features (payment history, hardship flags, settlement details) not available at prediction time
- **Column cleaning:** Removed administrative/ID columns, features with >10% missing values, and high-cardinality text fields
- **Feature engineering:** Converted string fields (term, interest rate, employment length, dates) into numeric form; applied TF-IDF to text where applicable

# PREPROCESSING - 2

- **Feature ablation:** Removed features shown in prior work to reduce model specificity
- **Bias prevention:** Removed Lending Club–assigned grading features (e.g., internal risk grades) to avoid built-in decision bias
- **Encoding:** One-hot encoded low-cardinality categorical variables
- **Train/Test split:** 70/30 time-ordered split (no shuffling) to prevent future data leakage
- **Scaling:** Applied MinMax scaling to  $[-1, 1]$ , fitted on training data only, for models like Logistic Regression and SVM



# METHOD 1: LOGISTIC REGRESSION

- Predicts **probability of non-default** using the **sigmoid function**.
- **Class-weighting** method used to account for class imbalance. Assigns higher weights to minority class.
- Architecture was tuned using **grid hyper parameter search** with **stratified holdout validation**. It consisted of:
  - **solver type:** newton, lbfgs
  - **regularization strength:** 0.01, 0.1, 1.0, 10
  - **class weight balancing:** on / off
- Final solver is **Newton**, **class balancing ON** and **regularization strength** of **10**, converged in 6 iterations. Good fit for smooth logistic-loss optimization.

## METHOD 2: NEURAL NETWORK

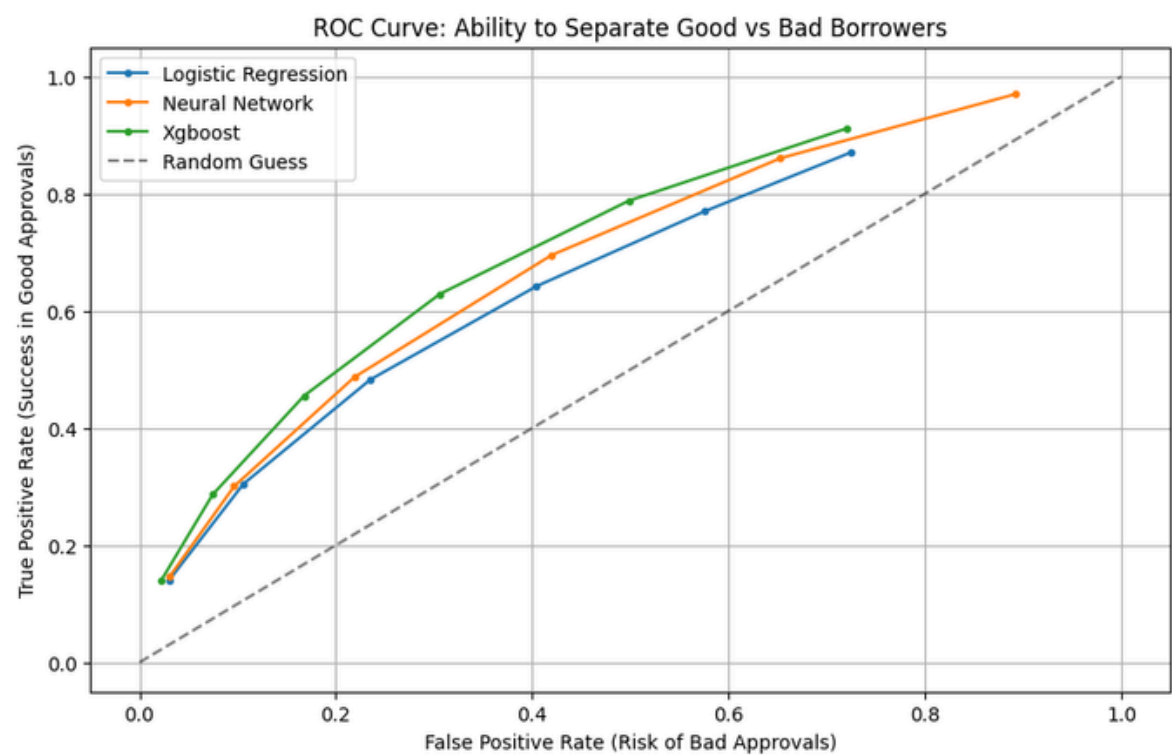
- A dense multilayer perceptron (MLP) was used for binary default prediction.
- Architecture was tuned using **manual hyper parameter search** with **stratified holdout validation**.
- Final selected model: **256 → 128 hidden** units with **ReLU activations** and **dropout = 0.2** after each dense layer.
- The output layer uses **sigmoid activation** to estimate the probability of the positive class, meaning non-default / fully repaid.
- The model was trained on scaled preprocessing features using **Adam**, **binary cross-entropy loss**, **class weights for imbalance**, and **early stopping** on validation AUC with **patience = 5**.

## METHOD 3: XG BOOST

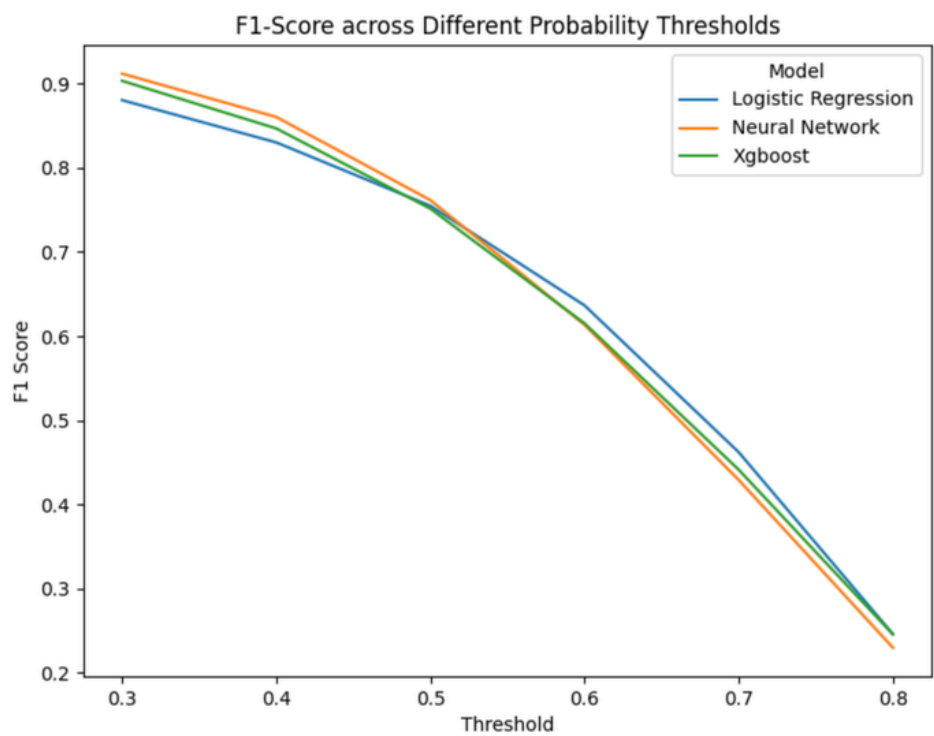
- A gradient-boosted tree ensemble (XGBoost) was used for binary default prediction.
- Final selected model: **tree splits** set to **6**, **minimum split loss** set to **5**, with **L1/L2 regularization**.
- The output is a **sigmoid probability** estimating the likelihood of the positive class (non-default / fully repaid).
- The model was trained on the unscaled preprocessed features using **histogram-based splitting**, **binary cross-entropy loss**, **class weights for imbalance**, and **early stopping** on validation AUC with patience = 40.



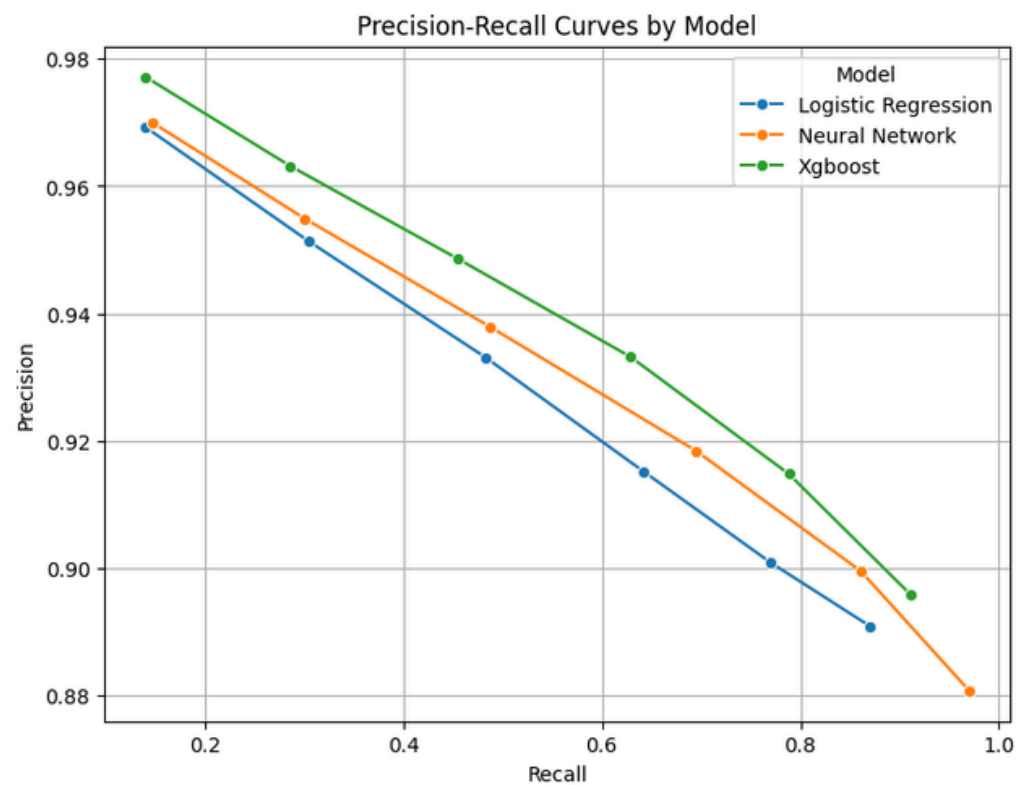
# RESULTS - 1



ROC Comparison for the three models



F1 Scores Across Different Thresholds



Precision-Recall Comparison for the three models

# CONFUSION MATRIX ANALYSIS

|                          | Logistic Regression | Neural Network | XGBoost |
|--------------------------|---------------------|----------------|---------|
| Caught Defaults (TN)     | 43,285              | 42,176         | 50,340  |
| Missed Defaults (FP)     | 29,393              | 30,502         | 22,248  |
| Rejected Good Loans (FN) | 176,755             | 150,568        | 183,230 |
| Approved Good Loans (TP) | 317,177             | 343,364        | 310,702 |

If **avoiding defaults** is most important, **XGBoost** is the strongest choice. If the goal is a better **balance between approving good borrowers** and **limiting bad approvals**, the **Neural Network** performs best at the 0.5 threshold.

## RESULTS - 2

- No single model dominates all metrics. The **Neural Network** has the **best F1 (0.762) and recall (64.7%) at threshold 0.5**, making it strongest for loan volume.
- **XGBoost** has the **highest ROC AUC (0.72), precision (93.3%), and specificity (69.4%)**, making it strongest for risk-averse lending and profit maximization.

**Is this sufficient to make business value propositions?**

# F1 THRESHOLDING

|   | Model               | threshold | accuracy | precision | recall   | f1_score | default_precision | default_recall | default_f1_score | approved_tp_fp | bad_approvals_fp |
|---|---------------------|-----------|----------|-----------|----------|----------|-------------------|----------------|------------------|----------------|------------------|
| 1 | Neural Network      | 0.3       | 0.842281 | 0.889006  | 0.935926 | 0.911863 | 0.320990          | 0.205853       | 0.250840         | 520001         | 57717            |
| 2 | Xgboost             | 0.3       | 0.830102 | 0.895867  | 0.910994 | 0.903367 | 0.316687          | 0.280346       | 0.297411         | 502272         | 52303            |
| 0 | Logistic Regression | 0.3       | 0.793985 | 0.890922  | 0.870215 | 0.880447 | 0.238278          | 0.275916       | 0.255719         | 482452         | 52625            |

- **Best F1 Threshold:** Chooses the threshold with the highest F1-score for each model. This favors overall balance between precision and recall and usually approves more loans.
- **Best F1 threshold** is **0.3** for all models. At 0.3, Neural Network leads with F1 0.912 and approves 520k loans, but **lets through 57,717 bad approvals**.

# CONSERVATIVE THRESHOLDING

|   | Model               | threshold | precision | recall   | f1_score | approved_tp_fp | bad_approvals_fp | default_precision | default_recall | default_f1_score |
|---|---------------------|-----------|-----------|----------|----------|----------------|------------------|-------------------|----------------|------------------|
| 0 | Logistic Regression | 0.7       | 0.951299  | 0.305222 | 0.462161 | 158477         | 7718             | 0.159164          | 0.893806       | 0.270210         |
| 1 | Neural Network      | 0.6       | 0.943241  | 0.454480 | 0.613404 | 237990         | 13508            | 0.180056          | 0.814139       | 0.294893         |
| 2 | Xgboost             | 0.6       | 0.948570  | 0.455223 | 0.615206 | 237040         | 12191            | 0.183533          | 0.832260       | 0.300745         |

- **Conservative Lending Threshold:** Limits **bad approvals** to **at most 15,000** and then chooses the threshold with the highest recall. This favors risk control while still approving as many good loans as possible under that bad-approval cap.
- **XGBoost** and **Neural Network** the best threshold is **0.6**, while **Logistic Regression** is **0.7**. XGBoost is the best conservative pick: 237k approved, only 12,191 bad approvals, 83.2% default recall.



# COST/PROFIT - 1

Threshold metrics alone are not enough because lending decisions are ultimately financial decisions.

We use the following assumptions for our analysis:

- **Each correctly approved non-default loan (TP) earns \$1,000.**
- **Each incorrectly approved default loan (FP) loses \$5,000.**
- Rejected loans are assigned no direct profit or loss because no loan is issued.

$$\text{expected\_profit} = (tp * \text{profit\_per\_good\_loan}) - (fp * \text{loss\_per\_bad\_loan})$$

# COST/PROFIT - 2

|   | Model               | threshold | approved_tp_fp | bad_approvals_fp | expected_profit | profit_per_approved_loan |
|---|---------------------|-----------|----------------|------------------|-----------------|--------------------------|
| 2 | Xgboost             | 0.4       | 425594         | 36250            | 208094000       | 488.949562               |
| 1 | Neural Network      | 0.4       | 445778         | 41498            | 196790000       | 441.452921               |
| 0 | Logistic Regression | 0.4       | 421988         | 41786            | 171272000       | 405.869361               |

- In profit analysis (\$1,000 per good loan, \$5,000 per default loss), **all models peak at threshold 0.4. XGBoost** produces the **highest expected profit (\$208.1M)**.
- This differs from the thresholding recommendations: The profit analysis shifts the preferred threshold upward to `0.4` because the cost of bad approvals is high enough that approving too many risky loans reduces total value.

# COST/PROFIT - 3

|   | Model               | profit_per_good_loan | loss_per_bad_loan | threshold | approved_tp_fp | bad_approvals_fp | expected_profit |
|---|---------------------|----------------------|-------------------|-----------|----------------|------------------|-----------------|
| 2 | Xgboost             | 1000                 | 3000              | 0.3       | 502272         | 52303            | 293060000       |
| 1 | Neural Network      | 1000                 | 3000              | 0.3       | 520001         | 57717            | 289133000       |
| 0 | Logistic Regression | 1000                 | 3000              | 0.3       | 482452         | 52625            | 271952000       |
| 5 | Xgboost             | 1000                 | 5000              | 0.4       | 425594         | 36250            | 208094000       |
| 4 | Neural Network      | 1000                 | 5000              | 0.4       | 445778         | 41498            | 196790000       |
| 3 | Logistic Regression | 1000                 | 5000              | 0.4       | 421988         | 41786            | 171272000       |
| 8 | Xgboost             | 1000                 | 10000             | 0.6       | 237040         | 12191            | 102939000       |
| 7 | Neural Network      | 1000                 | 10000             | 0.6       | 237990         | 13508            | 89402000        |
| 6 | Logistic Regression | 1000                 | 10000             | 0.7       | 158477         | 7718             | 73579000        |

- **Permissive** thresholds are **better for volume**, while **conservative** thresholds are better **when defaults are expensive** (\$3,000, \$5,000 and \$10,000). For a risk-aware lending strategy, XGBoost remains the strongest model because it gives the best profit under higher default-loss assumptions.



# FUTURE WORKS

- Improve interpretability using **SHAP (SHapley Additive exPlanations)** or **feature attribution** to explain which borrower and loan characteristics drive default-risk predictions.
- Test additional models such as **Random Forest, LightGBM, CatBoost**, or **ensemble methods** to see if ranking performance can improve beyond XGBoost.
- **Calibrate decision thresholds** around business goals, such as reducing bad approvals while preserving enough loan approvals.

Scan QR code to see  
GitHub repo and  
detailed results





**THANK YOU**